LIVEDESK

Self-Hosted Live Chat Platform

Complete technical system documentation outlining system architecture, technology stack, real-time message routing protocols, data schema models, and REST API reference endpoints.

Version: 1.0.0
Database: MongoDB
Runtime: Node.js / Express / Socket.io
Date: August 2026

■# LiveDesk — Project Documentation

> *A self-hosted live chat platform with real-time visitor messaging, agent dashboards, AI-powered fallback chatbot, and RAG-based knowledge retrieval.*

Table of Contents

What is LiveDesk?

LiveDesk is a **self-hosted live chat solution** that you embed into any website using a single `<script>` tag. It provides:

- ■ **Real-time visitor chat** via Socket.io
- ■■■ **Agent dashboard** to monitor visitors, take over chats, and reply live
- ■ **AI chatbot fallback** — when no agent is online or outside business hours, an AI bot answers using your knowledge base (RAG)
- ■ **Multi-project support** — each project has its own branding, settings, knowledge base, and chat sessions
- ■ **Business hours scheduling** — define working hours per project; the system auto-switches between human and bot mode

- ■ **Analytics dashboard** — live visitor counts, chat metrics, session history

---

Technology Stack

Layer	Technology	Purpose
Runtime	Node.js v22	JavaScript server runtime
Web Framework	Express.js v5	REST API and static file serving
Real-time	Socket.io v4	Bi-directional websocket communication
Database	MongoDB (via Mongoose v9)	Persistent data storage
AI / Embeddings	@xenova/transformers	Local ML model for generating text embeddings
Vector Search	In-memory cosine similarity (custom)	Knowledge base retrieval (RAG)
Authentication	JWT (jsonwebtoken) + bcryptjs	Agent login and session protection
Config	dotenvx	Environment variable management
Frontend	Vanilla HTML + CSS + JavaScript	Dashboard and widget UI (no framework)
Fonts	Google Fonts — Plus Jakarta Sans	Dashboard typography
Icons	Lucide Icons (CDN)	Dashboard UI icons

Project Structure

```
livedesk/
■■■ .env                        # Environment variables (secrets)
■■■ package.json                # Node dependencies
■■■ DOCUMENTATION.md            # This file
■
■■■ src/
■   ■■■ index.js                # Express server entry point, REST API routes
■   ■■■ socket.js               # All Socket.io real-time event handlers
■   ■
■   ■■■ config/
■   ■   ■■■ db.js               # MongoDB connection setup
■   ■
```

```
■   ■■■ models/                # Mongoose data schemas (MongoDB collections)
■   ■   ■■■ Agent.js           # Agent accounts (username, password, role)
■   ■   ■■■ Session.js         # Visitor chat sessions
■   ■   ■■■ Message.js         # Chat messages (visitor / agent / bot)
■   ■   ■■■ KBItem.js          # Knowledge base documents + embeddings
■   ■   ■■■ Setting.js         # Per-project settings (branding, hours)
■   ■
■   ■■■ services/
■       ■■■ rag.js             # RAG pipeline: embed, search, generate answer
■       ■■■ businessHours.js   # Business hours logic: fetch, validate, check open
■
■■■ public/                    # Static files served by Express
    ■■■ dashboard/
    ■   ■■■ index.html         # Agent dashboard SPA
    ■   ■■■ app.js             # Dashboard logic (auth, socket, rendering)
    ■   ■■■ style.css          # Dashboard styles (dark/light theme)
    ■
    ■■■ widget/
    ■   ■■■ widget.js          # Embeddable chat widget (Shadow DOM)
    ■   ■■■ widget.css         # Widget styles (scoped inside shadow DOM)
    ■
    ■■■ test-site/
        ■■■ index.html         # Sample website for testing the widget
```

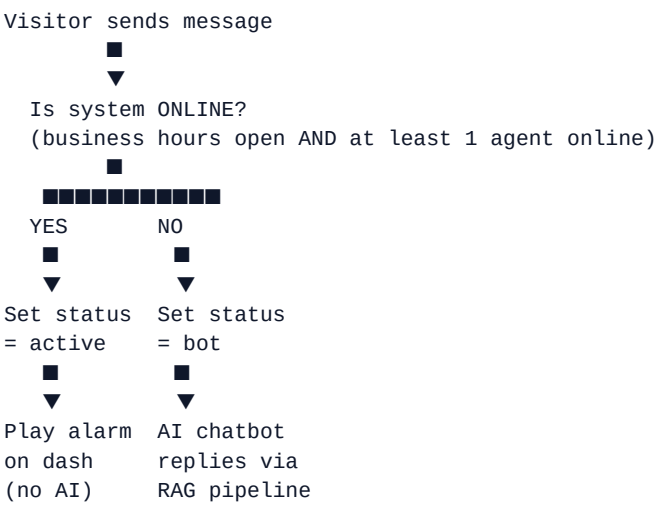
Architecture Overview

```
VISITOR BROWSER
  Any Website + <script src="/widget/widget.js?project=ID">
  ■■■ Shadow DOM Chat Widget
          ■ Socket.io WebSocket
          ▼
NODE.JS SERVER (port 3000)
  ■■■ Express REST API (/api/auth, /api/projects, /api/settings, /api/kb, /api/analytics)
  ■■■ Socket.io Engine
  ■   ■■■ visitor:register   → create/find session
  ■   ■■■ visitor:message    → route to human or AI
  ■   ■■■ agent:select_project → join project room
  ■   ■■■ agent:join_chat    → take over session
  ■   ■■■ notification:new_message → play alarm
  ■■■ RAG Pipeline
      ■■■ 1. Embed visitor query (local Xenova model)
      ■■■ 2. Cosine similarity search in KBItem docs
      ■■■ 3. Generate answer from matched chunks
          ■
          ▼
MONGODB (localhost:27017)
  Collections: agents, sessions, messages, kbitems, settings

AGENT DASHBOARD BROWSER
  localhost:3000
  ■■■ Login (JWT stored in localStorage)
  ■■■ Project Selector → switches socket room
```

```
  ■■■ Chats tab → live session list with unread badges
  ■■■ Knowledge Base tab → upload/delete documents
  ■■■ Settings tab → branding, business hours, agent management
  ■■■ Analytics tab → visitor metrics, session history
```

Message Routing Logic



Data Models (MongoDB Collections)

1. agents

Field	Type	Description
`_id`	ObjectId	Auto-generated unique ID
`username`	String	Agent login username
`password`	String	bcrypt hashed password
`role`	String	"admin" or "agent"
`isOnline`	Boolean	Online status toggle

2. sessions

Field	Type	Description
`_id`	ObjectId	Auto-generated unique ID
`visitorId`	String	Browser-generated visitor ID (e.g. "v_abc123")
`projectId`	String	Which project this session belongs to
`status`	String	"bot", "active", or "closed"
`assignedAgent`	ObjectId	Ref to agents (null in bot mode)
`unreadCount`	Number	Unread messages for agent
`visitorInfo.name`	String	From lead capture form
`visitorInfo.email`	String	From lead capture form
`visitorInfo.phone`	String	From lead capture form
`visitorInfo.currentPage`	String	Current URL visitor is on
`visitorInfo.ip`	String	Visitor IP address

3. messages

Field	Type	Description
`_id`	ObjectId	Auto-generated unique ID
`sessionId`	ObjectId	Ref to sessions
`sender`	String	"visitor", "agent", or "bot"
`text`	String	Message content
`timestamp`	Date	When message was sent

4. kbitems (Knowledge Base)

Field	Type	Description
`_id`	ObjectId	Auto-generated unique ID
`title`	String	Document name
`content`	String	Full text content
`projectId`	String	Scoped per project

Field	Type	Description
`embedding`	[Number]	384-dim float vector (Xenova model)

5. settings

Field	Type	Description
`_id`	ObjectId	Auto-generated unique ID
`key`	String	"widget_branding" or "business_hours"
`projectId`	String	Scoped per project
`value`	Mixed	Flexible JSON object

widget_branding value example:

```
{ "chatbotName": "Nora", "teamSubtitle": "Support Agent" }
```

business_hours value example:

```
{
  "enabled": true,
  "timezone": "Asia/Kolkata",
  "startTime": "09:00",
  "endTime": "18:00",
  "days": ["monday", "tuesday", "wednesday", "thursday", "friday"]
}
```

API Reference

All routes except `/api/auth/login` require: `Authorization: Bearer <token>`

Method	Route	Description
POST	/api/auth/login	Agent login → returns JWT token
POST	/api/auth/register	Register new agent (admin only)
GET	/api/projects	List all projects
POST	/api/projects	Create new project
GET	/api/kb?projectId=	List KB documents

Method	Route	Description
POST	/api/kb	Upload KB document
DELETE	/api/kb/:id	Delete KB document
GET	/api/settings/branding?projectId=	Get widget branding
POST	/api/settings/branding	Save widget branding
GET	/api/settings/business-hours?projectId=	Get business hours
POST	/api/settings/business-hours	Save business hours
GET	/api/analytics?projectId=&range=	Get analytics data

Socket Events Reference

Visitor → Server

Event	Payload	Description
visitor:register	{ projectId, visitorId, pageUrl, name, email }	Register and get session
visitor:message	{ text, sessionId }	Send chat message
visitor:typing	Boolean	Typing indicator
visitor:page_view	{ pageUrl, pageTitle }	Update current page
visitor:update_profile	{ name, email, phone }	Save lead form data

Server → Visitor

Event	Payload	Description
visitor:init	{ session, messages, branding }	Init widget with history
message:new	Message object	New message arrived
session:status_changed	{ status, assignedAgent }	Status changed
bot:typing	Boolean	AI typing indicator

Agent → Server

Event	Payload	Description
agent:select_project	{ projectId }	Subscribe to project room
agent:join_chat	{ sessionId }	Join/take over session
agent:message	{ sessionId, text }	Reply to visitor
agent:handoff_bot	{ sessionId }	Hand back to AI
agent:close_chat	{ sessionId }	Close session
agent:mark_read	{ sessionId }	Reset unread count

Server → Agent

Event	Payload	Description
sessions:init	Session[]	Initial session list
sessions:update	Session[]	Updated session list
agent:chat_history	{ sessionId, messages }	Full message history
notification:new_message	{ sessionId, visitorId, text }	New visitor message alert
visitor:typing_state	Boolean	Visitor is typing

Environment Variables

```
MONGODB_URI=mongodb://localhost:27017/livedesk
JWT_SECRET=your-super-secret-key-change-this
PORT=3000
```

Why MongoDB?

MongoDB is the **ideal choice for LiveDesk** for several key reasons:

1. Flexible Schema — Perfect for Chat

Chat data is variable. Visitor info may or may not have a name, email, or phone. Settings per project have completely different shapes. MongoDB handles this naturally without needing to ALTER TABLE or write migrations.

In SQL you would need either many nullable columns, a JSONB column, or complex joins for every variation.

2. Real-time Friendly

Socket.io delivers messages in real-time. MongoDB's document writes are fast for high-frequency inserts (chat messages, session updates) without the overhead of relational constraints.

3. Embedded Documents

MongoDB lets you store visitorInfo nested inside a Session document — one read, one write. In SQL this would require a separate visitor_info table with a foreign key join every time.

4. Flexible Value Storage

The Setting collection stores arbitrary JSON in its value field — a natural MongoDB pattern. This lets one collection hold branding configs, business hours, and any future setting type without schema changes.

5. Native Array/Vector Storage

The KBItem collection stores a 384-dimension float array (embedding vector) per document. MongoDB stores arrays natively. PostgreSQL requires the pgvector extension for this use case.

How to View Your MongoDB Data

Option 1: MongoDB Compass (Recommended — Free Official GUI)

This is the official MongoDB GUI — equivalent to pgAdmin for PostgreSQL.

1. Download from: https://www.mongodb.com/try/download/compass
2. Install and open Compass
3. Connect using: mongodb://localhost:27017
4. You will see the livedesk database with all collections
5. Browse documents (like rows in a table), filter, edit inline, and view indexes

Option 2: mongosh (Command Line Shell)

```
# Open the shell
mongosh

# Switch to livedesk database
use livedesk

# List all collections (like listing tables)
show collections

# View all sessions
db.sessions.find().pretty()

# View sessions for a specific project
db.sessions.find({ projectId: "project_wrwxyye" }).pretty()

# View all messages in a session
db.messages.find({ sessionId: ObjectId("paste_id_here") }).pretty()

# View all knowledge base items (title and project only)
db.kbitems.find({}, { title: 1, projectId: 1, _id: 0 }).pretty()

# View all settings
db.settings.find().pretty()

# Count documents
db.sessions.countDocuments()
db.messages.countDocuments()
```

Option 3: MongoDB for VS Code Extension

Install the MongoDB for VS Code extension (ID: mongodb.mongodb-vscode).

Connect to mongodb://localhost:27017 and browse collections in a tree view inside VS Code.

MongoDB vs PostgreSQL Concept Mapping

PostgreSQL	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
Primary Key (id)	_id (ObjectId)
Foreign Key	Reference field (ObjectId)

PostgreSQL	MongoDB
JOIN	$lookup aggregation or .populate()
SELECT * FROM sessions	db.sessions.find()
WHERE projectId = '...'	{ projectId: "..." }
UPDATE sessions SET ...	db.sessions.updateOne({ ... }, { $set: {...} })
pgAdmin	MongoDB Compass

Running the Project

```
# 1. Install dependencies
npm install

# 2. Make sure MongoDB is running locally
#    Download from https://www.mongodb.com/try/download/community

# 3. Start the server
node src/index.js

# 4. Open the dashboard
#    http://localhost:3000
#    Default login: admin / password123

# 5. Embed the widget in any website
#    <script src="http://localhost:3000/widget/widget.js?project=YOUR_PROJECT_ID"></script>
```

Default Seeding Account

When started with an empty database, a default admin is auto-seeded:

- Username: admin
- Password: password123
- Role: admin

Change this password immediately in production.

LiveDesk v1.0.0 — LiveDesk Platform Documentation Team